

安全原始碼

來源：<https://codelabs.developers.google.com/secure-source-code?hl=zh-tw>

步驟數：6

安全原始碼技術是一組做法，可用於提升原始碼的安全性。這些技術有助於識別及修正原始碼中的安全漏洞、防止未經授權的存取原始碼，並防止原始碼遭到修改。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. Linting](#)
4. [4. 靜態應用程式安全測試](#)
5. [5. 授權掃描](#)
6. [6. 恭喜](#)

1. 總覽

安全原始碼技術是一組實務做法，可用於提升原始碼的安全性。這些技術有助於找出並修正原始碼中的安全漏洞、防止未經授權存取原始碼，以及保護原始碼不遭修改。

常見的原始碼安全技術包括：

- **Linting**：Linting 是指檢查原始碼是否有錯誤和樣式問題的過程。這項作業會使用 Lint 工具完成，這類工具會分析原始碼並找出潛在問題。Lint 工具可用於檢查各種錯誤，包括語法錯誤、語意錯誤、樣式錯誤和安全漏洞。
- **靜態應用程式安全測試 (SAST)**：SAST 是一種安全測試，會分析原始碼、二進位碼或位元組碼，找出安全漏洞。SAST 工具可用於找出各種程式設計語言的安全性漏洞，包括 Go、Java、Python、C++ 和 C#。
- **授權掃描**：授權掃描是指識別軟體應用程式所用第三方軟體元件授權的程序。這很重要，因為有助於確保應用程式遵守授權條款，避免法律問題。

這些技術可用於在軟體開發生命週期的各個階段，提升原始碼的安全性。您可以在開發流程早期使用 Linting 找出錯誤，在編譯或部署程式碼前使用 SAST 找出安全漏洞，並使用授權掃描功能確保應用程式符合授權條款。

使用這些技術有助於提升原始碼的安全性，並降低安全漏洞的風險。

學習目標

本實驗室將著重於確保軟體原始碼安全無虞的工具和技術。

- Linting
- 靜態應用程式安全測試
- 授權掃描

本實驗室使用的所有工具和指令都會在 Cloud Shell 中執行。

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section follows, with a 'Project name' field containing 'my-kubernetes-codelab', also circled in blue. Below the name field, the 'Project ID' is displayed as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

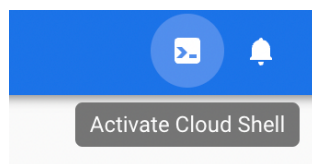
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格
4. 按一下「開啟編輯器」按鈕



5. 編輯器會開啟，右側顯示檔案總管，中央區域則顯示編輯器
6. 畫面底部也應顯示終端機窗格
7. 如果終端機尚未開啟，請使用 `ctrl+`` 鍵組合開啟新的終端機視窗

環境設定

將 GOPATH 設為單一目錄，簡化本實驗室中使用的指令。

```
export GOPATH=$HOME/gopath
```

建立目錄來保存我們的作業

```
mkdir -p workspace  
cd workspace
```

複製原始碼存放區

```
git clone https://gitlab.com/gcp-solutions-public/shift-left-security-workshop/source-code-lab.git  
cd source-code-lab  
export WORKDIR=$(pwd)
```

步驟 3 · 約 0 分鐘

3. Linting

Linting 可用來檢查常見的樣式錯誤或與語法相關的缺陷。Linting 可為多個團隊提供通用的語法模式，有助於加快程式碼審查速度、分享知識及提高程式碼的清晰度，進而協助確保安全性。

此外，Linting 會找出常見的語法錯誤，這些錯誤可能導致常見的安全性漏洞，例如不當或效率較低的程式庫或核心 API 用法。

安裝 `staticcheck` 連結工具

```
go get honnef.co/go/tools/cmd/staticcheck@latest
```

在專案根目錄中執行 Go Linter (staticcheck)

```
staticcheck
```

查看輸出內容。

```
main.go:42:29: unnecessary use of fmt.Sprintf (S1039)
```

您會收到錯誤訊息，是因為 `http.ListenAndServe()` 接受字串，而目前的程式碼使用 `Sprintf`，但未將變數傳遞至字串

查看指令結束狀態。

```
echo $?
```

在本例中，由於指令導致錯誤，結束狀態會是 **1** 或更大。這是 CI/CD 管道中可用來判斷工具成功/失敗的方法之一。

編輯 `main.go` 檔案，並修正程式碼：

- 在 `main()` 方法內的 `LINTING - Step 1` 下方，新增開頭斜線(`//`)，將該行設為註解予以排除。
- 移除開頭的斜線，取消註解 `main()` 方法中 `LINTING - Step 2` 下方的兩行程式碼。

在專案根目錄中重新執行 `staticcheck`

```
staticcheck
```

指令不應傳回任何結果 (即空白行)。

檢查指令的結束狀態。

```
echo $?
```

在本例中，由於指令未導致錯誤，結束狀態為**零**。

步驟 4 · 約 0 分鐘

4. 靜態應用程式安全測試

AST/靜態安全性測試 - 提供靜態程式碼分析，尋找常見的弱點和暴露 (CWE)

安裝 AST 工具 (gosec)

```
export GOSEC_VERSION="2.15.0"
curl -sL https://raw.githubusercontent.com/securego/gosec/master/install.sh | \
  sh -s -- -b $(go env GOPATH)/bin v${GOSEC_VERSION}
```

針對原始碼，使用政策檔案執行 gosec

```
gosec -conf policies/gosec-policy.json -fmt=json ./...
```

輸出內容應與以下所示相似

```
{
  "Golang errors": {},
  "Issues": [
    {
      "severity": "HIGH",
      "confidence": "LOW",
      "cwe": {
        "ID": "798",
        "URL": "https://cwe.mitre.org/data/definitions/798.html"
      },
      "rule_id": "G101",
      "details": "Potential hardcoded credentials",
      "file": "/home/random-user-here/shift-left-security-workshop/labs/source-code-lab/main.go",
      "code": "31: \\t// STEP 2: Change this and the reference below to something different (ie, not \\\"pawsword\\\" or \\\"password\\\")\\n32: \\tvar pawsword = \\\"im-a-cute-puppy\\\"\\n33: \\tfmt.Println(\\\"Something a puppy would use: \\\", username, pawsword)\\n",
      "line": "32",
      "column": "6"
    }
  ],
  "Stats": {
    "files": 1,
    "lines": 89,
    "nosec": 0,
    "found": 1
  }
}
```

這項工具發現潛在問題： Potential hardcoded credentials

步驟 5 · 約 0 分鐘

5. 授權掃描

授權對安全性很重要，因為授權可能會合法要求您公開可能不想公開的原始碼。這個概念稱為「[著作權保留](#)」授權，如果您使用具有這類授權的依附元件，就必須公開原始碼。

安裝「golicense」

```
mkdir -p /tmp/golicense
wget -O /tmp/golicense/golicense.tar.gz
https://github.com/mitchellh/golicense/releases/download/v0.2.0/golicense_0.2.0_linux_x86_64.
tar.gz
pushd /tmp/golicense
tar -xzf golicense.tar.gz
chmod +x golicense
mv golicense $(go env GOPATH)/bin/golicense
popd
```

建構二進位檔案

```
go build
```

使用不允許「BSD-3-Clause」授權的現行政策檔案執行授權檢查

```
golicense policies/license-policy.hcl hello-world
```

注意：這項操作應該會失敗，並顯示類似以下的輸出內容：

```
❌ rsc.io/sampler      BSD 3-Clause "New" or "Revised" License
❌ rsc.io/quote        BSD 3-Clause "New" or "Revised" License
❌ golang.org/x/text   BSD 3-Clause "New" or "Revised" License
```

修改政策檔案 `policies/license-policy.hcl`，將「BSD-3-Clause」從 `deny` 清單移至 `allow` 清單。

重新執行授權檢查

```
golicense policies/license-policy.hcl hello-world
```

注意：這項操作應會成功，並輸出類似下列內容：

```
✅ rsc.io/quote        BSD 3-Clause "New" or "Revised" License
✅ rsc.io/sampler      BSD 3-Clause "New" or "Revised" License
✅ golang.org/x/text   BSD 3-Clause "New" or "Revised" License
```

如要在實際工作環境中使用這些指令 (以及以原始碼為目標的其他工具)，請將其併入 CI/CD 管道。

步驟 6 · 約 0 分鐘

6. 恭喜

恭喜，您已完成本程式碼研究室！

您學到的內容

- 保護原始碼的工具和技術

—

上次更新時間：2023 年 3 月 23 日
